

编译原理讨论题及答案整理

按题目前编号顺序整理，可直接复制使用

目录

- 【讨论1-1】编译过程与人工翻译过程之间的关系
- 【讨论1-2】符号表中NAME字段的设计
- 【讨论2-1】计算机中是如何表示语言的？
- 【讨论2-2】每一类单词可否看成一种语言？
- 【讨论4-1】关于 FOLLOW 集的计算方法
- 【讨论4-2】下推自动机与有穷自动机之间的关系
- 【讨论4-3】LR语法分析栈中状态信息与文法符号信息之间的关系
- 【讨论4-4】为什么合并同心项集不会产生移进-归约冲突？
- 【讨论4-5】为什么LALR分析法不会作错误的移进操作？
- 【讨论5-1】如何表示语义信息？
- 【讨论5-2】如何计算语义属性？
- 【讨论5-3】如何判定一个属性是综合属性还是继承属性？
- 【讨论5-4】什么样的SDD可以保证存在对其属性进行计算的顺序？
- 【讨论5-5】为什么S-SDD与L-SDD能够保证存在属性计算顺序？
- 【讨论5-6】为什么S-SDD的SDT可以在语法分析的过程中同时实现？
- 【讨论5-7】非递归预测分析器的扩展
- 【讨论5-8】递归预测分析器的扩展
- 【讨论5-9】文法及翻译方案的修改
- 【讨论6-1】如何计算数组的类型表达式及数组宽度？
- 【讨论6-2】数组元素寻址的翻译
- 【讨论7-1】如何基于符号表构造访问链？
- 【讨论7-2】如何基于符号表访问非局部数据？
- 【讨论7-3】语言的过程递归调用和过程嵌套声明特性会影响编译器的哪些问题？

【讨论1-1】编译过程与人工翻译过程之间的关系

题目：编译过程与人工翻译过程之间的关系

编译过程和人工翻译过程有一定的相似性，本质上都是把一种语言表达的内容转换成另一种语言表达的内容。人工翻译时，译者通常先阅读原文，识别单词和句子结构，理解句子的含义，再按照目标语言的表达习惯重新组织并输出译文。编译过程也是类似的：编译程序首先进行词法分析，把源程序分解成一个个单词符号；然后进行语法分析，判断这些单词是否组成合法的语法结构；接着进行语义分析，检查类型、作用域等语义信息是否正确；之后生成中间代码并进行优化，最后生成目标代码。二者的区别在于，人工翻译面对的是自然语言，允许一定的灵活性和语境理解，而编译面对的是程序设计语言，规则更加严格，必须按照形式化的词法、语法和语义规则进行处理，不能随意解释。可以说，编译过程是一种高度形式化、自动化的翻译过程，它把高级语言程序翻译成计算机能够执行或进一步处理的低级语言程序。

【讨论1-2】符号表中NAME字段的设计

题目：符号表中NAME字段的设计

符号表中的 NAME 字段用于记录标识符的名字，它是编译程序查找和区分不同标识符的重要依据。在设计 NAME 字段时，不能简单地在每个符号表项中都直接存放完整字符串，因为标识符长度不固定，如果直接定长存储会浪费空间，长度不够又会造成截断问题。因此，常见做法是把标识符的实际字符串统一存放在一个字符串表中，而符号表的 NAME 字段只保存指向该字符串的指针或在字符串表中的入口地址。这样既可以支持不同长度的标识符，又能节省符号表空间，提高管理效率。查找符号时，可以通过 NAME 字段找到对应的标识符字符串，再结合作用域、类型、种属、存储位置等信息判断它的具体含义。对于不同作用域中同名的标识符，NAME 字段可以相同，但它们在符号表中是不同表项，需要依靠作用域层次等字段加以区分。因此，NAME 字段的设计既要能够准确表示标识符名称，又要便于快速查找和节省存储空间。

【讨论2-1】计算机中是如何表示语言的？

题目：计算机中是如何表示语言的？

计算机中表示语言，通常不是直接表示“自然语言的含义”，而是把语言看成一定字符集上的字符串集合，并用形式化方法描述这些字符串的组成规则。在编译原理中，一个程序设计语言可以看成由字符、单词、语句和程序等层次组成的形式语言。最底层用字符集表示基本符号，例如字母、数字和运算符；词法层面用正规式或正规文法描述各类单词符号，如标识符、常数、关键字等，并可用有穷自动机识别；语法层面用上下文无关文法描述语句和程序结构，如表达式、赋值语句、循环语句等，并用语法分析器判断输入程序是否符合语法规则。因此，计算机中表示语言的核心方法是用文法、正规式、自动机等形式化工具，把语言表示为一组满足规则的符号串集合，从而使编译程序能够对源程序进行识别、分析和翻译。

【讨论2-2】每一类单词可否看成一种语言？

题目：每一类单词可否看成一种语言？

每一类单词都可以看成一种语言。因为在编译原理中，语言本质上是某个字母表上符合一定规则的字符串集合，而每一类单词也正好可以看作一组具有共同形式特征的字符串集合。例如，标识符可以看成由字母、数字等组成且满足“不能以数字开头”等规则的字符串集合；整数常量可以看成由数字组成的集合；关键字可以看成由 if、while、return 等固定字符串组成的有限集合；运算符和界符也可以看成由特定符号组成的集合。因此，每一类单词都对应一种形式语言，并且通常可以用正规式来描述，用有穷自动机来识别。词法分析的过程，本质上就是判断输入字符序列中的各个子串分别属于哪一类单词语言，并把它们转换成相应的单词符号。

【讨论4-1】关于 FOLLOW 集的计算方法

题目：关于 FOLLOW 集的计算方法

这里蓝色字体写成“FOLLOW(A) 中的所有符号都在 FOLLOW(B) 中”，是因为在产生式 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta$ 且 β 可以推出空串 ϵ 的情况下，B 后面已经没有固定的符号，或者 β 可以消失，所以 B 后面可能直接跟着原来跟在 A 后面的符号。也就是说，凡是能够出现在 A 后面的终结符，在这种情况下也都能够出现在 B 后面，因此应当把 FOLLOW(A) 加入到 FOLLOW(B) 中，也就是 FOLLOW(A) 包含于 FOLLOW(B)。不能反过来说“FOLLOW(B) 中的所有符号都在 FOLLOW(A) 中”，因为 B 可能还会出现在其他产生式中，或者在某些产生式中后面直接跟着终结符，这些符号也会进入 FOLLOW(B)，但它们不一定能跟在 A 后面。例如有文法 $S \rightarrow A$, $A \rightarrow B | Bc$ ，则 \$ 在 FOLLOW(A) 中，也会进入 FOLLOW(B)；同时由于 $A \rightarrow Bc$ ，c 也在 FOLLOW(B) 中，但 c 不在 FOLLOW(A) 中。所以只能说 FOLLOW(A) 的符号加入 FOLLOW(B)，而不能说 FOLLOW(B) 的符号都属于 FOLLOW(A)。

【讨论4-2】下推自动机与有穷自动机之间的关系

题目：下推自动机与有穷自动机之间的关系

下推自动机和有穷自动机都是形式语言中的识别模型，它们的共同点是都由状态、输入符号和转移规则组成，都可以根据输入串的扫描过程判断该串是否属于某种语言。二者的主要区别在于是否具有额外的存储能力。有穷自动机只有有限个状态，没有辅助存储结构，因此只能记住有限的信息，主要用于识别正则语言，例如词法分析中的单词符号识别。下推自动机在有穷自动机的基础上增加了一个栈，可以通过压栈和出栈保存和匹配信息，因此具有更强的识别能力，能够识别上下文无关语言，例如语法分析中的括号匹配、嵌套结构和程序语句结构。可以认为，有穷自动机是下推自动机的一种特殊情况：当下推自动机不使用栈，或者栈不起实际作用时，它的能力就退化为有穷自动机。因此，下推自动机比有穷自动机表达能力更强，二者分别对应编译过程中的词法分析和语法分析两个重要阶段。

【讨论4-3】LR语法分析栈中状态信息与文法符号信息之间的关系

题目：LR语法分析栈中状态信息与文法符号信息之间的关系

在LR语法分析中，分析栈中通常同时保存状态信息和文法符号信息，它们是一一对应、相互配合的关系。文法符号表示当前已经识别出的语法成分，例如终结符、非终结符；状态信息则表示在识别到这些文法符号之后，分析器所处的LR自动机状态，也可以理解为当前已经识别了哪些产生式右部前缀，以及接下来可能期待什么符号。一般来说，栈的形式可以看作“状态—符号—状态—符号—状态”交替出现，其中每个文法符号下面的状态表示移进或归约前的状态，而文法符号后面的状态表示经过该文法符号转移后到达的状态。移进时，分析器把当前输入符号压入栈，同时根据ACTION表得到的新状态也压入栈；归约时，按照产生式右部长度弹出相应数量的文法符号和状态，再把产生式左部非终结符压入栈，并根据栈顶状态和该非终结符查GOTO表得到新状态。由此可见，文法符号记录了“已经识别出的语法结构”，状态信息记录了“识别这些结构后分析器的控制位置”，二者共同决定下一步是移进、归约、接受还是报错。

【讨论4-4】为什么合并同心项集不会产生移进-归约冲突？

题目：为什么合并同心项集不会产生移进-归约冲突？

合并同心项集不会产生移进-归约冲突，是因为同心项集具有相同的LR(0)核心，合并时只是把相同核心项目的向前看符号集合合并起来，并不会改变项目中圆点的位置，也不会改变圆点后面的文法符号。移进动作由形如 $A \rightarrow \alpha \cdot a \beta$ 的项目决定，只要圆点后面是终结符 a ，在输入符号为 a 时就可能移进，而这一点只与LR(0)核心有关，与向前看符号无关。因此，同心项集合并后不会新增移进项目，也不会新增移进动作。假如合并后出现了移进-归约冲突，说明合并后的项集中既有一个对 a 的移进项目，又有一个在向前看符号 a 上的归约项目。由于被合并的项集核心相同，那么包含该归约项的原LR(1)项集中也一定包含那个移进项目，这样原来的LR(1)项集本身就已经存在移进-归约冲突了，这与原LR(1)分析表无冲突相矛盾。所以，合并同心项集不会产生新的移进-归约冲突。合并可能带来的问题主要是归约-归约冲突，因为不同项集中的归约项目向前看符号合并后可能发生重叠。

【讨论4-5】为什么LALR分析法不会作错误的移进操作？

题目：为什么LALR分析法不会作错误的移进操作？

LALR分析法不会作错误的移进操作，原因在于LALR是由LR(1)项目集族合并得到的，它只合并具有相同LR(0)核心的项目集，而移进动作只由LR(0)核心决定。也就是说，只要某个项目中圆点后面是终结符，例如 $A \rightarrow \alpha \cdot a \beta$ ，分析器在遇到输入符号 a 时就可能移进，这个判断与LR(1)项目的向前看符号无关。LALR合并项目集时，只是把相同核心项目的向前看符号集合合并起来，并不会改变项目的核心，也不会改变圆点后面的符号和状态转移关系。因此，LALR表中的移进动作和原来的LR(1)分析表中的移进动作是一致的，不会因为合并项目集而新增错误的移进动作。合并可能带来的问题主要出现在归约动作上，因为向前看符号集合扩大后，可能在某些符号上增加归约动作，从而产生归约-归约冲突，或者使错误发现稍微推迟。但移进动作本身不依赖向前看符号集合的扩大，所以LALR分析法不会作错误的移进操作。

【讨论5-1】如何表示语义信息？

题目：如何表示语义信息？

语义信息通常通过属性和语义规则来表示。在编译原理中，语法规则只能描述程序结构是否正确，而语义信息用来描述这些结构所具有的含义，例如表达式的类型和值、变量在符号表中的位置、数组的宽度、语句生成的中间代码等。为了表示这些信息，可以给文法符号附加属性，属性分为综合属性和继承属性；同时在产生式上定义语义规则，用来说明这些属性应该如何计算。例如在表达式 $E \rightarrow E1 + T$ 中，可以用属性 $E.type$ 表示表达式类型，用 $E.val$ 表示表达式的值，也可以用语义规则说明 $E.val = E1.val + T.val$ 。因此，语义信息的表示方法就是在上下文无关文法的基础上，为文法符号增加属性，并用语义规则或语义动作描述属性之间的关系，从而把语法结构和程序含义联系起来。

【讨论5-2】如何计算语义属性？

题目：如何计算语义属性？

计算语义属性时，首先要根据语法制导定义中给出的语义规则，确定每个属性依赖哪些其他属性，然后按照属性之间的依赖关系选择合适的计算顺序。一般来说，可以先为分析树建立属性依赖图，图中的结点表示属性，边表示某个属性的值依赖另一个属性的值；如果依赖图无环，就可以通过拓扑排序得到属性计算顺序。在实际编译过程中，综合属性通常由子结点传递给父结点，因此可以在自底向上的过程中计算；继承属性通常由父结点或左兄弟结点传递给当前结点，因此常在自顶向下、从左到右的过程中计算。对于 S-SDD，由于只有综合属性，可以在归约时直接计算；对于 L-SDD，由于继承属性的依赖受到限制，可以在递归下降分析过程中一边分析一边传递和计算属性。因此，语义属性的计算本质上就是根据语义规则和属性依赖关系，保证每个属性在被使用之前已经计算出来。

【讨论5-3】如何判定一个属性是综合属性还是继承属性？

题目：如何判定一个属性是综合属性还是继承属性？

判定一个属性是综合属性还是继承属性，关键要看这个属性值是从哪里得到的。如果一个文法符号的属性值主要由它的子节点属性计算得到，也就是信息从语法树的下层向上层传递，那么这个属性就是综合属性。例如在表达式求值中，一个表达式的值通常由它的各个子表达式的值计算得到，这就是综合属性。相反，如果一个文法符号的属性值是由它的父节点、兄弟节点或者上下文环境传递下来的，也就是信息从语法树的上层或旁边传向该节点，那么这个属性就是继承属性。例如变量声明中的类型信息，可以由声明语句传递给各个变量名，这种类型属性就是继承属性。因此，判断时可以看属性依赖关系：依赖子节点的是综合属性，依赖父节点或兄弟节点的是继承属性。

【讨论5-4】什么样的SDD可以保证存在对其属性进行计算的顺序？

题目：什么样的SDD可以保证存在对其属性进行计算的顺序？

能够保证存在属性计算顺序的 SDD，通常要求它的属性依赖关系不能形成循环，也就是说它的依赖图必须是无环的。如果一个属性的计算依赖另一个属性，而另一个属性又反过来依赖它，就会出现循环依赖，这样就无法确定谁先计算，属性计算顺序也就不存在。因此，只要一个 SDD 的依赖图是有向无环图，就可以通过拓扑排序得到一个合法的属性计算顺序。在常见的语法制导定义中，S-SDD 和 L-SDD 都能保证存在属性计算顺序。S-SDD 只包含综合属性，属性值从子节点向父节点传递，可以按照语法树自底向上的顺序计算；L-SDD 对继承属性的依赖作了限制，要求继承属性只能依赖父节点属性和左边符号的属性，因此可以按照从左到右、深度优先的顺序计算。总的来说，只要属性之间没有循环依赖，并且每个属性在被使用前都已经能够被计算出来，这样的 SDD 就能保证存在属性计算顺序。

【讨论5-5】为什么S-SDD与L-SDD能够保证存在属性计算顺序？

题目：为什么S-SDD与L-SDD能够保证存在属性计算顺序？

S-SDD 和 L-SDD 能够保证存在属性计算顺序，是因为它们都对属性之间的依赖关系作了限制，使属性不会出现无法确定先后顺序的循环依赖。S-SDD 中只包含综合属性，综合属性只依赖于子节点的属性，因此可以按照语法树自底向上的顺序计算：先计算叶子节点和子节点的属性，再计算父节点的属性，这样一定能够保证每个属性在被使用之前已经计算出来。L-SDD 虽然既可以有综合属性，也可以有继承属性，但它规定一个符号的继承属性只能依赖产生式左部的继承属性，以及它左边符号的属性，不能依赖右边尚未分析到的符号。因此，L-SDD 可以按照从左到右、深度优先的顺序计算属性，特别适合递归下降或预测分析过程。也就是说，S-SDD 依靠“自底向上”的依赖关系保证计算顺序，L-SDD 依靠“从左到右”的依赖限制保证计算顺序，所以二者都能避免属性循环依赖，从而保证属性计算是可实现的。

【讨论5-6】为什么S-SDD的SDT可以在语法分析的过程中同时实现？

题目：为什么S-SDD的SDT可以在语法分析的过程中同时实现？

S-SDD 的 SDT 可以在语法分析过程中同时实现，是因为 S-SDD 只包含综合属性，而综合属性的值只依赖于当前结点的子结点属性，也就是属性信息是从语法树的叶子结点向根结点自底向上传递的。在语法分析过程中，当一个产生式右部的各个符号都已经分析完成后，它们的属性值也就已经可以确定，此时就可以立即执行该产生式对应的语义动作，计算左部非终结符的综合属性。因此，S-SDD 的语义动作通常可以放在产生式的末尾，在归约时执行。特别是在自底向上的 LR 分析中，每次按照某个产生式进行归约时，右部符号的属性已经在栈中保存好了，分析器可以直接取出这些属性，计算左部符号的属性并压回栈中。由于不需要依赖父结点或右兄弟结点的信息，所以不会出现属性尚未计算却被使用的问题。因此，S-SDD 的 SDT 很适合与语法分析过程结合，在完成语法分析的同时完成属性计算和语义处理。

【讨论5-7】非递归预测分析器的扩展

题目：非递归预测分析器的扩展

为了在非递归预测分析过程中实现 L-属性定义的 SDT，需要把原来只存放文法符号的分析栈扩展为可以同时存放文法符号、属性值和语义动作的栈。由于非递归预测分析是用显式分析栈模拟递归下降过程的，所以继承属性不能再通过函数参数传递，综合属性也不能再通过函数返回值返回，而是要通过栈中的属性记录来传递和保存。通常可以为每个文法符号在栈中建立一个记录，记录该符号的综合属性、继承属性以及指向相关属性位置的指针；同时把语义动作也作为一种特殊符号压入栈中，等它出现在栈顶时就执行相应动作。在分析过程中，当根据预测分析表选择产生式 $A \rightarrow X_1 X_2 \dots X_n$ 时，不仅要把产生式右部符号按逆序压栈，还要在适当位置插入语义动作，用来计算各符号的继承属性或综合属性。对于 L-属性定义来说，每个符号的继承属性只依赖左部符号的继承属性和它左边符号的属性，因此在从左到右分析时，这些属性都可以在需要前通过栈中记录计算出来。终结符匹配时取得词法属性，非终结符展开前先准备它的继承属性，子结构分析完成后再通过语义动作计算综合属性并传回相关记录。这样，非递归预测分析器就可以通过扩展分析栈和在栈中安排语义动作，实现对 L-属性定义的语法制导翻译。

【讨论5-8】递归预测分析器的扩展

题目：递归预测分析器的扩展

为了在递归预测分析过程中实现 L-属性定义的语法制导翻译，需要把原来的递归下降分析器从“只识别语法结构”扩展为“边分析边计算属性并执行语义动作”的形式。具体来说，可以把每个非终结符对应的分析过程看成一个函数：继承属性作为该函数的参数从父节点或左兄弟节点传入，综合属性作为函数的返回值传出。在函数内部，根据预测分析表或当前输入符号选择相应产生式，然后按照产生式右部从左到右依次分析各个文法符号，并在适当位置插入语义动作。由于 L-属性定义中，每个符号的继承属性只能依赖产生式左部的继承属性和它左边符号的属性，所以这种从左到右的递归预测分析顺序正好可以保证属性在使用前已经被计算出来。对于终结符，可以在匹配输入符号时取得它的词法属性；对于非终结符，则在调用其分析过程前先计算并传入继承属性，调用结束后接收其综合属性。这样，递归预测分析器就可以在完成语法分析的同时完成类型检查、中间代码生成、符号表填写等翻译任务。

【讨论5-9】文法及翻译方案的修改

题目：文法及翻译方案的修改

如果想用自底向上的方法完成原来可以用自顶向下方式完成的翻译过程，需要把翻译方案改造成适合归约时执行语义动作的形式。因为自底向上分析中，语义动作通常只能在产生式右部已经识别完成、发生归约时执行，而自顶向下翻译中有些语义动作可能出现在产生式右部中间，用来提前计算继承属性或传递上下文信息。为了解决这个问题，可以在文法中引入一些新的“标记非终结符”或“动作非终结符”，把原来嵌在产生式中间的语义动作改写成一个产生空串的产生式。例如原来有 $A \rightarrow \alpha \{动作\} \beta$ ，可以改写为 $A \rightarrow \alpha M \beta$ ，并增加 $M \rightarrow \epsilon$ ，把原来的语义动作放在 $M \rightarrow \epsilon$ 的归约动作中。这样，当自底向上分析器识别完 α 后，就可以归约出 M ，从而执行原来位于中间的语义动作。修改后的问题是，引入空产生式 and 新的非终结符后，可能会改变原文法的分析性质，导致自底向上分析表中出现新的冲突，例如移进-归约冲突；同时文法和语义动作也会变得更复杂。这个问题在一定条件下是可以解决的，例如通过合理设计标记非终结符、调整文法结构、推迟某些语义动作的执行，或者利用语义栈保存中间属性值来实现。但并不是所有翻译方案都能机械地改写后仍保持无冲突，因此实际编译器中通常要结合具体文法和属性依赖关系来判断是否适合用自底向上的方法实现。

【讨论6-1】如何计算数组的类型表达式及数组宽度？

题目：如何计算数组的类型表达式及数组宽度？

对数组声明语句进行翻译时，需要根据数组的维数、各维长度和数组元素的基本类型来计算数组的类型表达式和数组宽度。数组的类型表达式通常可以表示为 $array(n, T)$ ，其中 n 表示该维数组的元素个数， T 表示数组元素的类型；如果是多维数组，则可以递归表示，例如 $array(n1, array(n2, int))$ 表示一个二维整型数组。数组宽度是整个数组在存储空间中所占的字节数，计算方法是“元素个数 $\times$ 元素类型宽度”。对于一维数组 $A[n]$ of T ，若类型 T 的宽度为 w ，则数组宽度为 $n \times w$ ；对于多维数组，则从最内层的基本类型开始向外逐层计算，例如 $A[n1][n2]$ of int ，若 int 的宽度为 4，则内层数组类型为 $array(n2, int)$ ，宽度为 $n2 \times 4$ ，外层数组类型为 $array(n1, array(n2, int))$ ，总宽度为 $n1 \times n2 \times 4$ 。因此，数组声明翻译的过程就是先确定基本类型及其宽度，再根据每一维的长度递归构造数组的类型表达式，同时逐层累乘计算数组所需的总存储空间，并把数组名、类型表达式和宽度等信息填入符号表。

【讨论6-2】数组元素寻址的翻译

题目：数组元素寻址的翻译

对一个数组元素进行寻址时，需要知道数组的首地址、数组元素的宽度、数组的维数、各维的上下界以及数组在内存中的存放方式，例如按行优先还是按列优先。同时还需要知道下标表达式的值，因为数组元素的地址本质上是由“数组首地址 + 偏移量”计算出来的。对于一维数组，若数组为 $A[l..u]$ ，元素宽度为 w ，则 $A[i]$ 的地址可以翻译为 $\text{base}(A) + (i - l) * w$ ；如果下界为 0，则简化为 $\text{base}(A) + i * w$ 。对于多维数组，需要根据存储顺序把多个下标转换成一个线性偏移量。例如按行优先存储的二维数组 $A[l1..u1, l2..u2]$ ，元素 $A[i, j]$ 的偏移量为 $((i - l1) * n2 + (j - l2)) * w$ ，其中 $n2$ 是第二维的长度。因此，数组元素寻址的翻译过程就是先翻译各个下标表达式，计算出它们对应的偏移量，再乘以元素宽度，最后加上数组的基地址，生成相应的中间代码或目标代码来完成地址计算。

【讨论7-1】如何基于符号表构造访问链？

题目：如何基于符号表构造访问链？

基于符号表构造访问链，关键是利用符号表中记录的过程嵌套层次和作用域关系。编译程序在处理过程声明时，会为每个过程或分程序建立相应的符号表，并在符号表中记录该过程的静态层次、外层过程以及局部变量的偏移量等信息。这样在生成过程调用代码时，就可以根据“调用过程所在层次”和“被调用过程所属外层层次”的关系，确定被调用过程活动记录中的访问链应该指向哪一个外层活动记录。访问链通常也叫静态链，它指向该过程在源程序静态结构中的直接外层过程的最新活动记录。如果被调用过程是当前过程的内层过程，则它的访问链可以直接指向当前过程的活动记录；如果调用的是同层或外层过程中的过程，则需要沿当前活动记录的访问链向外追溯若干层，找到被调用过程的直接外层过程对应的活动记录，再把它作为新活动记录的访问链。符号表在这里提供的是静态作用域层次和层次差，运行时活动记录中的访问链负责把这些静态层次关系落实到具体的存储地址上。因此，基于符号表构造访问链的过程，本质上就是根据过程的嵌套关系确定每次调用时新活动记录应连接到哪一个外层活动记录，从而保证内层过程能够正确访问非局部变量。

【讨论7-2】如何基于符号表访问非局部数据？

题目：如何基于符号表访问非局部数据？

基于符号表访问非局部数据，核心是利用符号表中保存的作用域层次信息和存储位置信息来确定变量真正所在的活动记录。编译程序在处理一个变量名时，会先在当前作用域的符号表中查找，如果找不到，就沿着外层作用域的符号表继续查找，直到找到该变量的声明。符号表项中通常会记录变量所属的过程或分程序层次、变量在该活动记录中的偏移量、类型和宽度等信息。生成目标代码时，如果该变量不是当前过程的局部变量，就需要根据当前过程和变量声明过程之间的层次差，找到外层过程对应的活动记录，再根据偏移量访问该变量。实现时常用的方法有静态链和显示表：静态链是在每个活动记录中保存一个指向直接外层活动记录的指针，访问非局部变量时沿静态链向外追溯若干层；显示表则直接保存各层最新活动记录的地址，可以更快定位某一层的活动记录。因此，符号表负责确定“变量在哪一层、偏移是多少”，运行时的静态链或显示表负责找到对应活动记录，二者结合就可以实现对非局部数据的正确访问。

【讨论7-3】语言的过程递归调用和过程嵌套声明特性会影响编译器的哪些问题？

题目：语言的过程递归调用和过程嵌套声明特性会影响编译器的哪些问题？

语言的过程递归调用和过程嵌套声明特性，会直接影响编译器对运行时存储管理、名字作用域管理和非局部变量访问方式的处理。过程递归调用意味着同一个过程在运行时可能同时存在多个活动记录，因此编译器不能只为一个过程分配固定的静态存储区，而需要采用栈式存储管理，每调用一次过程就在运行时栈中建立一个新的活动记录，用来保存局部变量、参数、返回地址、控制链等信息，过程返回时再释放对应的活动记录。过程嵌套声明则会带来静态作用域层次问题，即内层过程可以访问外层过程中的变量，因此编译器在符号表中不仅要记录标识符的名字、类型和偏移量，还要记录它所在的作用域层次。生成代码时，如果访问的是非局部变量，就需要根据符号表中的层次差，通过静态链或显示表找到外层过程的活动记录，再根据偏移量取得变量。因此，递归调用主要影响活动记录的动态分配和运行时栈管理，嵌套声明主要影响作用域分析、符号表组织以及非局部数据访问机制，二者共同决定了编译器如何设计过程调用、参数传递、返回控制和存储访问。